

HW2 - Part 1: Movies!

✓ Submission Instructions

1) First create a copy of this notebook in your drive and rename it so that it contains the course number (ISTA322), the semester (Sp22 for Spring 2022, Su22 for Summer 2022, and Fa22 for Fall 2022), the assignment code (HW1 for this assignment) and your name.

e.g. my copy for Spring 2023 would be ISTA_322_Sp23_HW2_Part1_dan_charbonneau and ISTA_3

2) When you are ready to submit. Prepare three files for each part (total of 6 files): the python file (File->Download->Download .py), the notebook file (File->Download->Download .ipynb), and **PDF** version of your notebook (**after running all cells**). You should be able to create a pdf through File -> print -> Destination: save as pdf

3) Create a new folder named firstname_lastname_hw1 (e.g my folder would be dan_charbonneau_hw2) put all six files in it. Compress (Zip) the folder you created with the files inside of it and submit this .zip file to D2L

incorrect filenames or submission formats will result in a loss of 50% of your grade

✓ Wrangling and aggregating movie review data

The website MovieLens.com has a research group which provides open access to millions of reviews, for free! We're going to work with those data for this homework. [Feel free to check out the website here](#). You can go and download the raw data, but in order to make things a bit easier, the files have been uploaded the Google drive for fast direct downloads.

There are two datasets we'll be working with.

- movies - this is a file of 60,000+ movies
- reviews - this is a file of 25 *million* individual reviews for the 60k movies

The goal for this section of the homework is to do two types of data aggregations that will allow for someone to make inferences on which movies were the most popular, reviewed, polarizing and were cult classics.

To do this we'll first start by making a simple data set that brings just overall review properties together with movies. We'll then do some deeper groupings to create a dataset that looks at the

together with movies. We'll then do some deeper groupings to create a dataset that looks at the same properties but over time.

▼ Data first

Let's bring in our two files and libraries. The ratings file is understandably large. So it's a good idea to download it and then save a copy as something else and work with that. This way if you mess up you don't have to download it all over again

```
# Libraries
import pandas as pd
from datetime import datetime
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Movies data
movies = pd.read_csv('https://itsa322.s3.us-east-2.amazonaws.com/movies.csv')
```

movies

	movieId	title	genres	
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	
1	2	Jumanji (1995)	Adventure Children Fantasy	
2	3	Grumpier Old Men (1995)	Comedy Romance	
3	4	Waiting to Exhale (1995)	Comedy Drama Romance	
4	5	Father of the Bride Part II (1995)	Comedy	
...	...	...	...	
62418	209157	We (2018)	Drama	
62419	209159	Window of the Soul (2001)	Documentary	
62420	209163	Bad Poems (2018)	Comedy Drama	
62421	209169	A Girl Thing (2001)	(no genres listed)	
62422	209171	Women of Devil's Island (1962)	Action Adventure Drama	

Next steps: [Generate code with movies](#) [View recommended plots](#) [New interactive sheet](#)

```
# Ratings data
ratings = pd.read_csv('https://itsa322.s3.us-east-2.amazonaws.com/ratings.csv')

# Make a copy of ratings; Generally, it's good practice to keep original copies saved bef
# Also if you want to re-do some of the cells you can use rating_backup instead of loadin
ratings_backup = ratings.copy
```

✓ Q1 Explore your data - [3 points]

Below take some time to explore your date.

In one cell, check the following items for **both** datasets:

- Head and tail
- Shape
- Datatypes

In a new cell:

- The total number of NaN values for the movies and ratings tables (a single value for each table)

Task Do the head,tail, and shape operations all in one cell. Count the number of NaNs in another.

```
## Q1 Your code starts here
# shape of movies
print("movies shape:", ratings.shape );
```

```
movies shape: (25000095, 4)
```

```
#head of movies
ratings.head()
```

	userId	movieId	rating	timestamp	
0	1	296	5.0	1147880044	
1	1	306	3.5	1147868817	
2	1	307	5.0	1147868828	
3	1	665	5.0	1147878820	
4	1	899	3.5	1147868510	

```
#tail of movies
ratings.tail()
```

	userId	movieId	rating	timestamp	
25000090	162541	50872	4.5	1240953372	
25000091	162541	55768	2.5	1240951998	
25000092	162541	56176	2.0	1240950697	
25000093	162541	58559	4.0	1240953434	
25000094	162541	63876	5.0	1240952515	

```
# Get the total count of NaNs for movies
number_of_NaN_in_movies = ratings['movieId'].isna().sum() #replace 0 here with your code
print("Number of NaNs in movies:", number_of_NaN_in_movies)
```

```
Number of NaNs in movies: 0
```

```
#Do the same steps for shape, head, and tail, and NaNs of ratings
...
```

```
number_of_NaN_in_ratings = ratings['rating'].isna().sum() #replace 0 here with your code
print("Number of NaNs in ratings:", number_of_NaN_in_ratings)
## Q1 Your code ends here - Any code outside of these start/end markers won't be graded
```

```
Number of NaNs in ratings: 0
```

✓ Q2 Convert timestamp in ratings to a datetime. - [1.5 points]

One issue that you can see from your exploration is that the ratings only have a timestamp. This timestamp is measured in the number of seconds since 00:00:00 on January 1st, 1970. You'll need to convert this to a datetime in order to actually do our later data aggregations.

You use `pd.to_datetime` on timestamps like this. [For full details on the various ways to use this function please look at the Pandas documentation.](#) Briefly, this is how it's used.

```
>>> pd.to_datetime(1490195805, unit='s')
Timestamp('2017-03-22 15:16:45')
```

Task Make a new column in the ratings dataframe called `review_dt` that contains the data from the timestamp column but converted to a datetime datatype.

Note it's good practice to do this by first assigning the output to a test vector first rather than

note, its good practice to do this by first assigning the output to a test vector, `test_vec`, rather than directly adding the output to the dataframe. This allows you to make sure your operation did what you wanted before modifying your dataframe. Once you're confident of the output, you can assign the vector to the `reviews` dataframe as a new column.

For example

```
test_vec = pd.to_datetime(arguments)
test_vec # to check what it contains
```

Q2 Your code starts here

Make test vector

```
review_dt = pd.to_datetime(ratings['timestamp'], unit='s')
```

Check vector and datatype to make sure it make sense.

```
review_dt.head()
```

	timestamp
0	2006-05-17 15:34:04
1	2006-05-17 12:26:57
2	2006-05-17 12:27:08
3	2006-05-17 15:13:40
4	2006-05-17 12:21:50

dtype: datetime64[ns]

Now add this to the ratings dataframe

```
ratings['review_dt'] = review_dt
```

Check the head of your dataframe again

```
ratings.head()
```

	userId	movieId	rating	timestamp	review_dt
0	1	296	5.0	1147880044	2006-05-17 15:34:04
1	1	306	3.5	1147868817	2006-05-17 12:26:57
2	1	307	5.0	1147868828	2006-05-17 12:27:08
3	1	665	5.0	1147878820	2006-05-17 15:13:40
4	1	899	3.5	1147868510	2006-05-17 12:21:50

```
# What's the oldest and newest review? It should be 1995-01-09 and 2019-11-21, respectively
oldest_review = ratings['timestamp'].min()
print(oldest_review)
```

```
newest_review = ratings['timestamp'].max()
print(newest_review)
```

```
## Q2 Your code ends here - Any code outside of these start/end markers won't be graded
```

```
789652009
1574327703
```

✓ Q3 Your first aggregation and join - [4.5 points]

The first aggregation and join I want you to do is at the whole movie level. Your movies dataframe should only have a single row for each movie, but there are obviously thousands of individual reviews for each of those movies. Our goal here is to produce some summary statistics about the reviews for each movie and then join them to the movies dataframe.

Task: Do the following:

- Create an aggregated dataframe called `ratings_by_movie`. This dataframe should be grouped by movie. Use `.agg()` to calculate the mean, standard deviation ('std'), and the number of reviews for each movie.
- Rename the columns of that dataframe `'movieId'`, `'rating_mean'`, `'rating_std'`, and `'rating_count'`
- Join this new `ratings_by_movie` dataframe such that it attaches all those summary statistics to their corresponding movies from the `'movies'` dataframe.
- Call the joined dataframe `movies_with_ratings`

```
## Q3 Your code starts here
```

```
# Make ratings_by_movie
```

```
ratings_by_movie = ratings.groupby('movieId', as_index=False)['rating'].agg(['mean', 'std', 'count'])
```

```
# Check it
```

```
ratings_by_movie.head()
```

	movieId	mean	std	count
0	1	3.893708	0.921552	57309
1	2	3.251527	0.959851	24228
2	3	3.142028	1.008443	11804



3	4	2.853547	1.108531	2523
---	---	----------	----------	------

4	5	3.058434	0.996611	11714
---	---	----------	----------	-------

Next
steps:

[Generate code with ratings_by_movie](#)
[View recommended plots](#)
[New interactive sheet](#)

```
# Rename columns
new_col_names = ['movieId', 'rating_mean', 'rating_std', 'rating_count']
ratings_by_movie = ratings_by_movie.copy()
ratings_by_movie.columns = new_col_names

# Join it and call movies with ratings
movies_with_ratings = pd.merge(movies, ratings_by_movie, on='movieId', how='left')

# Check movies_with_ratings.
movies_with_ratings.shape
## Q3 Your code ends here - Any code outside of these start/end markers won't be graded

(62423, 6)
```

Is your merged dataframe 62423 rows $\times$ 6 columns?

✓ Q4 Filtering and more transformations - [3 points]

Now we want to clean up this dataset a bit and then do a couple more transforms. One issue you can see from your check above is that many movies only have one rating. We're going to choose to set a minimum number of reviews needed to be included. We also want to do some binning where movies with certain ratings levels

Task: Please do the following operations

- Filter `movies_with_ratings` so it only contains movies that have at least 10 ratings
- Use the function `cut()` to automatically bin our `rating_mean` column into three groups of 'bad', 'fine', or 'good' movies. Call this `rating_group`.
- Use the same function to take the standard deviation in rating and make three groups of 'agreement', 'average', 'controversial'. Thus, movies with low standard deviation have agreement in the rating, while movies with high standard deviation have controversy in the ratings. Call this column `ratings_agreement`.

```
## Q4 part 1 Your code starts here
# Filter first being sure to overwrite dataframe
movies_with_ratings = movies_with_ratings[movies_with_ratings['rating_count'] >= 10]
```

```
movies_with_ratings = movies_with_ratings[movies_with_ratings['rating_count'] >= 10]

# Check how many rows you're left with. You should have a little over 24000
movies_with_ratings.shape
## Q4 part 1 Your code ends here - Any code outside of these start/end markers won't be g

(24330, 6)
```

I didn't show you how to use `cut()` in the lesson, but it's a transform just like anything else. You could use a `np.where()` statement like we did, but `cut()` is a bit easier. All it does is take 1) a column as the first argument, 2) the number of bins you want to group it in as the second argument, and then 3) the labels you want to give those bins as the third. It automatically divides them up into equal sized bins.

For example, if I make the following list:

```
rating = [2, 4, 9, 8, 5, 3, 6, 10, 2, 1, 6, 7]
```

And run `cut()` on it with three bins and levels 'bad', 'fine', and 'good':

```
pd.cut(rating, 3, labels=['bad', 'fine', 'good'])
```

I get a return of:

```
[bad, bad, good, good, fine, ..., good, bad, bad, fine, fine]
Length: 12
Categories (3, object): [bad < fine < good]
```

Note how it orders them for you based on the order of the labels.

```
# If you want to test it!
rating = [2, 4, 9, 8, 5, 3, 6, 10, 2, 1, 6, 7]
pd.cut(rating, 3, labels=['bad', 'fine', 'good'])

['bad', 'bad', 'good', 'good', 'fine', ..., 'good', 'bad', 'bad', 'fine', 'fine']
Length: 12
Categories (3, object): ['bad' < 'fine' < 'good']
```

Note: if get a `SettingWithCopyWarning` that's ok here. Pandas tries to warn us that we are working with a copy of `movies_with_rating`, and the changes won't reface back to the original `movies_with_rating` dataframe. But that's ok for us

```
## Q4 part 2 Your code starts here
# Now make 'bad', 'fine', 'good' levels for ratings
# Assign to new column called 'rating_group'

movies_with_ratings['rating_group'] = pd.cut(movies_with_ratings['rating_mean'], 3, label

# Check it
movies_with_ratings
```

	movieId	title	genres	rating_mean	rating_std	rating_count	rating
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	3.893708	0.921552	57309.0	
1	2	Jumanji (1995)	Adventure Children Fantasy	3.251527	0.959851	24228.0	
2	3	Grumpier Old Men (1995)	Comedy Romance	3.142028	1.008443	11804.0	
3	4	Waiting to Exhale (1995)	Comedy Drama Romance	2.853547	1.108531	2523.0	
4	5	Father of the Bride Part II (1995)	Comedy	3.058434	0.996611	11714.0	

Next steps:

[Generate code with movies_with_ratings](#)
[View recommended plots](#)
[New interactive sl](#)

Do Toy Story and Jumanji have 'good' ratings? Does Grumpier Old Men have a 'fine' rating?

```
# Now use cut() again to create your ratings_agreement column.
# Use three bins and order of 'agreement', 'average', and 'controversial'
movies_with_ratings['ratings_agreement'] = pd.cut(movies_with_ratings['rating_std'], 3, 1

# Check to make sure that your bin categories make sense. e.g. 'good' movies should have
movies_with_ratings.head()
## Q4 part 2 Your code ends here - Any code outside of these start/end markers won't be g
```

movieId	title	genres	rating_mean	rating_std	rating_count	rating_group
		Adventure				

0	1	Toy Story (1995)	Animation Children Comedy Fantasy	3.893708	0.921552	57309.0	good
1	2	Jumanji (1995)	Adventure Children Fantasy	3.251527	0.959851	24228.0	good

✓ Exploring our data

Making bins like this allows us to figure out things like which movies are both bad, but have differing opinions on. For example, we can filter out movies that are in the bad category but have a lot of controversy about those ratings. This could mean they're 'cult classic' movies where despite the low rating some people actually really love the movies.

A dataset like this could be used in a recommendation engine where if you see people liking these 'bad but good' movies you could suggest others that meet the same criteria.

Task: There are no points here - only code to let you make figures to see what the dataset could be used for. Also, if this code works it means you probably did your answers above are right :)

```
movies_with_ratings[(movies_with_ratings['rating_group'] == 'bad') &
                    (movies_with_ratings['ratings_agreement'] == 'controversial') &
                    (movies_with_ratings['rating_count'] >= 100)]
```

	movieId	title	genres	rating_mean	rating_std	rating_count	rating_group
8618	26157	Manos: The Hands of Fate	Horror	1.798165	1.550268	109.0	bad

If you have done everything correctly so far, you will get: The Hands of Fate (1966) Horror and Birdemic: Shock and Terror (2010) as the outcome.

✓ Q5 Grouping within years - [3 point]

Now that we've done our overall grouping by movie, let's get a bit more detail about these ratings. Specifically, let's engineer a dataset that breaks down the average rating not only by movie, but also by the year the person provided the review. This would allow for someone to see which movies continue to do well over time, which ones become more popular, and which ones don't age well!

Task: You're going to do the following steps:

- Create a new `ratings_by_movie_year` that groups both by `movieId` but also by your `review_dt` column. I want you to group into year intervals.
- Join `movies` to `ratings_by_movie_year` so that you have the summary review statistics for each year the movie has been out
- Clean up and filter your dataframe

First, create `ratings_by_movie_year`. You can group by two levels by just adding a list of what levels you want to group by in the `groupby()` statement. I'll give you some help there, but you have to complete the rest in order to group by `movieId` first and `review_dt` second.

```
# Note the groupby syntax. I first am grouping by movieId
# But then also am calling dt.year on our datetime column
# This will then tell Python to do the aggregations within year as well

## Q5 Your code starts here
ratings_by_movie_year = ratings.groupby([ratings['movieId'], ratings['review_dt'].dt.year

# Check
ratings_by_movie_year.head()
```

		mean	std	count	
movieId	review_dt				
1	1996	4.132756	0.884990	6237	
	1997	3.872500	0.894079	6000	
	1998	3.889515	0.944972	887	
	1999	3.975140	0.913960	2494	
	2000	4.136634	0.865127	3535	

You need to rename columns, but this is a bit trickier as you have two levels of your dataframe index. I'm going to give you the code below. But, what it's doing is resetting that one level of the index `review_dt` and putting it back as a regular column. I'm then renaming the resulting columns.

```
# Reset index (you cannot rerun this cell (or any cell that includes reset_index) twice u
ratings_by_movie_year = ratings_by_movie_year.reset_index(level='review_dt')
ratings_by_movie_year.columns = ['year', 'rating_mean', 'rating_std', 'rating_count']
ratings_by_movie_year.head()
```

	year	rating_mean	rating_std	rating_count	
movieId					
1	1996	4.132756	0.884990	6237	
1	1997	3.872500	0.894079	6000	
1	1998	3.889515	0.944972	887	
1	1999	3.975140	0.913960	2494	
1	2000	4.136634	0.865127	3535	

```
# Now join the movie dataframe onto ratings_by_movie_year. Think about the type/direction
# and how that will affect what data will be kept and what data will be dropped. Think ab
# that don't have corresponding data in the other table, etc
ratings_by_movie_year = pd.merge(movies, ratings_by_movie_year, on='movieId', how='left')
```

```
# Check
ratings_by_movie_year.head()
```

	movieId	title	genres	year	rating_mean	rating_std	rating_count	
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1996.0	4.132756	0.884990	6237.0	
1	1	Toy Story (1995)	Adventure Animation Children Comedy	1997.0	3.872500	0.894079	6000.0	

```
# How many rows are there in the resulting dataframe?
print(ratings_by_movie_year.shape)
## Q5 Your code ends here - Any code outside of these start/end markers won't be graded

(327113, 7)
```

In your dataframe do you see Toy Story having a mean rating of 4.132756 in 1996? Is your dataframe 323737 rows $\times$ 7 columns?

✓ A quick plot

Now you have a dataset where one could explore how movies have done over time. I've made a couple plots below to show you what I mean.

```
# What movie has the max rating?
```

```
movies_with_ratings.loc[movies_with_ratings['rating_count'] == movies_with_ratings['rating_count'].max]
```

	movieId	title	genres	rating_mean	rating_std	rating_count	rating_group
7927	356	Forrest Gump (1994)	Comedy Drama Romance War	4.120873	0.931022	15264.0	Good

If you have done everything correctly you should get Forrest Gump (1994)

```
# Grabbing just the ID for that movie.
```

```
movies_with_ratings['movieId'][movies_with_ratings['rating_count'] == movies_with_ratings['rating_count'].max]
```

```
np.int64(356)
```

```
# Forest gump has been reviewed over 80000 times!
```

```
# Let's extract that movie id to an object and then make just that dataframe
```

```
most_viewed_id = movies_with_ratings['movieId'][movies_with_ratings['rating_count'] == movies_with_ratings['rating_count'].max]
```

```
most_viewed_df = ratings_by_movie_year[ratings_by_movie_year['movieId'] == most_viewed_id]
most_viewed_df
```

	movieId	title	genres	year	rating_mean	rating_std	rating_count	
7927	356	Forrest Gump (1994)	Comedy Drama Romance War	1996.0	4.120873	0.931022	15264.0	
7928	356	Forrest Gump (1994)	Comedy Drama Romance War	1997.0	4.085352	0.919867	1816.0	
7929	356	Forrest Gump (1994)	Comedy Drama Romance War	1998.0	4.093245	1.046018	1051.0	
7930	356	Forrest Gump (1994)	Comedy Drama Romance War	1999.0	3.787516	1.163514	3172.0	
7931	356	Forrest Gump (1994)	Comedy Drama Romance War	2000.0	4.027959	1.051722	3827.0	
7932	356	Forrest Gump (1994)	Comedy Drama Romance War	2001.0	4.119985	0.993906	2667.0	

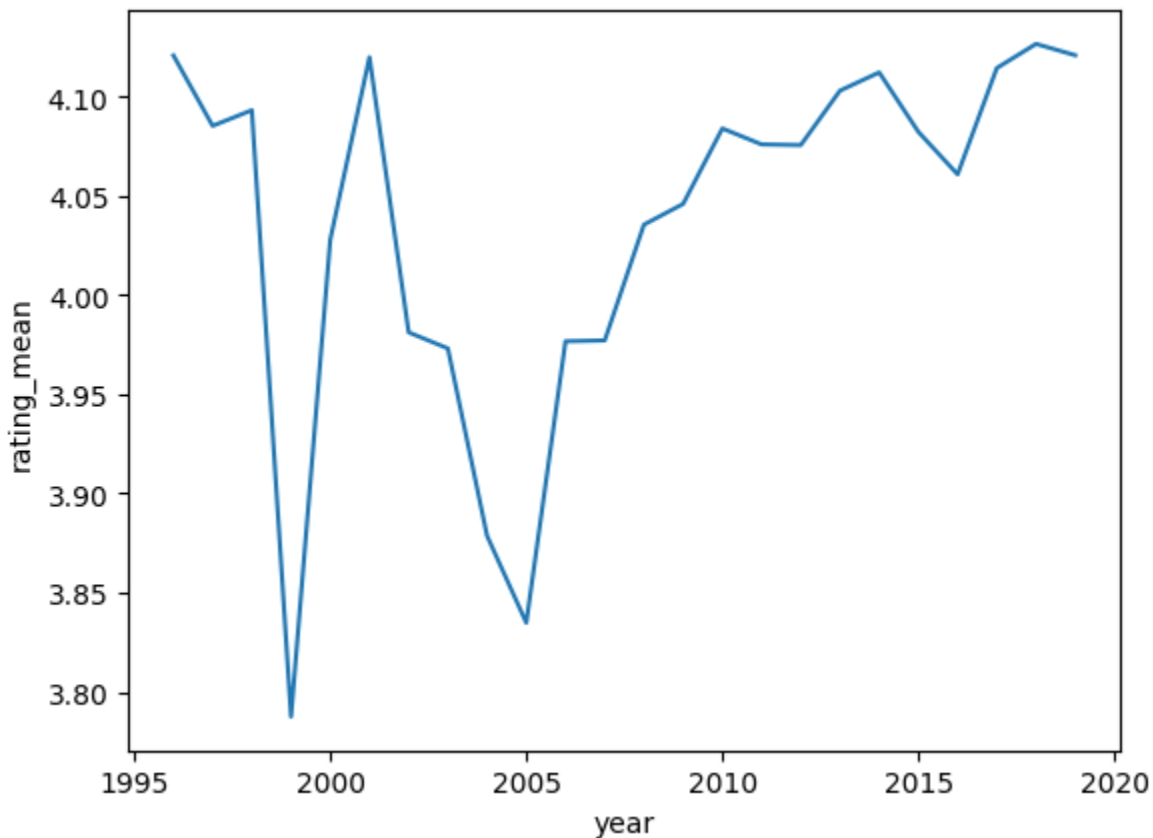
			vvar				
7933	356	Forrest Gump (1994)	Comedy Drama Romance War	2002.0	3.981262	1.014716	1601.0
7934	356	Forrest Gump (1994)	Comedy Drama Romance War	2003.0	3.973144	0.973239	1657.0
7935	356	Forrest Gump (1994)	Comedy Drama Romance War	2004.0	3.878659	0.951275	2596.0

Next steps:

[Generate code with most_viewed_df](#)
[View recommended plots](#)
[New interactive sheet](#)

```
# A quick lineplot shows that although forest gump has a really high rating on average,
sns.lineplot(data = most_viewed_df, x = 'year', y = 'rating_mean')
```

<Axes: xlabel='year', ylabel='rating_mean'>



You can also look up some of your favorite movies. I actually love the movie 'Dredd', even though many people hated it. We can search in the titles for movies we like, and then call that ID to filter a new dataframe. We can then make some plots

```
# Search for Dredd
movies_with_ratings[movies_with_ratings['title'].str.contains('Dredd')]

# Call the dredd to make a dataframe
dredd_id = movies_with_ratings['movieId'][movies_with_ratings['title'].str.contains('^Dre
dredd_df = ratings_by_movie_year[ratings_by_movie_year['movieId'] == dredd_id]
dredd_df
```

Next steps: [Generate code with dredd_df](#) [View recommended plots](#) [New interactive sheet](#)

```
# Our plot shows that the year after release it was reviewed pretty poorly,
# but the score gradually grew as people realized how awesome it was :)
sns.lineplot(x = 'year', y = 'rating_mean', data = dredd_df)
```

